Generics: Generische Klassen & Methoden

Carsten Gips (HSBI)

TL;DR
Generische Klassen und Methoden sind ein wichtiger Baustein in der Programmierung mit Java. Dabei werden Typ-Variablen eingeführt, die dann bei der Instantiierung der generischen Klassen oder beim Aufruf von generischen Methoden mit existierenden Typen konkretisiert werden ("Typ-Parameter").
Syntaktisch definiert man die Typ-Variablen in spitzen Klammern hinter dem Klassennamen bzw. vor dem Rückgabetyp einer Methode: `public class Stack<E> { }` und `public <T> T foo(T m){ }`.

Videos

- VL Generische Klassen & Methoden
- Demo Generische Methoden

Unterlagen

- PDF-Version

---

1 Generische Strukturen

```
Vector speicher = new Vector();
speicher.add(1); speicher.add(2); speicher.add(3);
speicher.add("huhu");

int summe = 0;
for (Object i : speicher) { summe += (Integer)i; }
```

Problem: Nutzung des "*raw*" Typs `Vector` ist nicht typsicher!

- Mögliche Fehler fallen erst zur Laufzeit und u.U. erst sehr spät auf: Offenbar werden im obigen Beispiel `int`-Werte erwartet, d.h. das Hinzufügen von `"huhu"` ist vermutlich ein Versehen (wird vom Compiler aber nicht bemerkt)
- Die Iteration über `speicher` kann nur allgemein als `Object` erfolgen, d.h. in der Schleife muss auf den vermuteten/gewünschten Typ gecastet werden: Hier würde dann der String `"huhu"` Probleme zur Laufzeit machen

---

```
Vector<Integer> speicher = new Vector<Integer>();
speicher.add(1); speicher.add(2); speicher.add(3);
speicher.add("huhu");

int summe = 0;
for (Integer i : speicher) { summe += i; }
```

Vorteile beim Einsatz von Generics:

- Datenstrukturen/Algorithmen nur einmal implementieren, aber für unterschiedliche Typen nutzen
- Keine Vererbungshierarchie nötig
- Nutzung ist typsicher, Casting unnötig

- Geht nur für Referenztypen
- Beispiel: Collections-API

2 Generische Klassen/Interfaces definieren

- **Definition**: "`<Typ>`" hinter Klassennamen

```
public class Stack<E> {
    public E push(E item) {
        addElement(item);
        return item;
    }
}
```

  - `Stack<E>` => Generische (parametrisierte) Klasse (auch: "*generischer Typ*")
  - `E` => Formaler Typ-Parameter (auch: "*Typ-Variable*")

- **Einsatz**:

```
Stack<Integer> stack = new Stack<Integer>();
```

  - `Integer` => Typ-Parameter
  - `Stack<Integer>` => Parametrisierter Typ

3 Generische Klassen instantiieren

- Typ-Parameter in spitzen Klammern hinter Klasse bzw. Interface

```
ArrayList<Integer> il = new ArrayList<Integer>();
ArrayList<Double>  dl = new ArrayList<Double>();
```

4 Beispiel I: Einfache generische Klassen

```
class Tutor<T> {
    // T kann in Tutor *fast* wie Klassenname verwendet werden
    private T x;
    public T foo(T t) { ... }
}
```

```
Tutor<String>  a = new Tutor<String>();
Tutor<Integer> b = new Tutor<>();  // ab Java7: "Diamond Operator"

a.foo("wuppie");
```

```
5  b.foo(1);
6  b.foo("huhu");  // Fehlermeldung vom Compiler
```

Beispiel: classes.GenericClasses

4.1 Typ-Inferenz

Typ-Parameter kann bei `new()` auf der rechten Seite oft weggelassen werden => **Typ-Inferenz**

```
1  Tutor<String> x = new Tutor<>();  // <>: "Diamantoperator"
```

(gilt seit Java 1.7)

5 Beispiel II: Vererbung mit Typparametern

```
1  interface Fach<T1, T2> {
2      public void machWas(T1 a, T2 b);
3  }
4
5  class SHK<T> extends Tutor<T> { ... }
6
7  class PM<X, Y, Z> implements Fach<X, Z> {
8      public void machWas(X a, Z b) { ... }
9      public Y getBla() { ... }
10 }
11
12 class Studi<A,B> extends Person { ... }
13 class Properties extends Hashtable<Object,Object> { ... }
```

Auch Interfaces und abstrakte Klassen können parametrisierbar sein.

Bei der Vererbung sind alle Varianten bzgl. der Typ-Variablen denkbar. Zu beachten ist dabei vor allem, dass die Typ-Variablen der Oberklasse (gilt analog für Interfaces) entweder durch Typ-Variablen der Unterklasse oder durch konkrete Typen spezifiziert sind. Die Typ-Variablen der Oberklasse dürfen nicht “in der Luft hängen” (siehe auch nächste Folie)!

6 Beispiel III: Überschreiben/Überladen von Methoden

```
1  class Mensch { ... }
2
3  class Studi<T extends Mensch> {
4      public void f(T t) { ... }
5  }
6
7  class Prof<T> extends Mensch { ... }
8
9  class Tutor extends Studi<Mensch> {
10     public void f(Mensch t) { ... }      // Ueberschreiben
```

```
    public void f(Tutor t) { ... }       // Ueberladen
}
```

7 Vorsicht: So geht es nicht!

```
class Foo<T> extends T { ... }

class Fluppie<T> extends Wuppie<S> { ... }
```

- Generische Klasse `Foo<T>` kann nicht selbst vom Typ-Parameter `T` ableiten (warum?)
- Bei Ableiten von generischer Klasse `Wuppie<S>` muss deren Typ-Parameter `S` bestimmt sein: etwa durch den Typ-Parameter der ableitenden Klasse, beispielsweise `Fluppie<S>` (statt `Fluppie<T>`)

8 Generische Methoden definieren

- "`<Typ>`" vor Rückgabetyp

```
public class Mensch {
    public <T> T myst(T m, T n) {
        return Math.random() > 0.5 ? m : n;
    }
}
```

- "Mischen possible":

```
public class Mensch<E> {
    public <T> T myst(T m, T n) { ... }
    public String myst(String m, String n) { ... }
}
```

9 Aufruf generischer Methoden

9.1 Aufruf

- Aufruf mit Typ-Parameter vor Methodennamen, oder
- Inferenz durch Compiler

9.2 Finden der richtigen Methode durch den Compiler

1. Zuerst Suche nach exakt passender Methode,
2. danach passend mit Konvertierungen => Compiler sucht gemeinsame Oberklasse in Typhierarchie

9.3 Beispiel

```
class Mensch {
    <T> T myst(T m, T n) { ... }
}
Mensch m = new Mensch();


m.<String>myst("Essen", "lecker");  // Angabe Typ-Parameter


m.myst("Essen", 1);          // String, Integer => T: Object
m.myst("Essen", "lecker");   // String, String  => T: String
m.myst(1.0, 1);              // Double, Integer => T: Number
```

Beispiel methods.GenericMethods

Reihenfolge der Suche nach passender Methode gilt auch für nicht-generisch überladene Methoden

```
class Mensch {
    public <T> T myst(T m, T n) {
        System.out.println("X#myst: T");
        return m;
    }

    // NICHT gleichzeitig erlaubt wg. Typ-Löschung (s.u.):
/*
    public <T1, T2> T1 myst(T1 m, T2 n) {
        System.out.println("X#myst: T");
        return m;
    }
*/

    public String myst(String m, String n) {
        System.out.println("X#myst: String");
        return m;
    }

    public int myst(int m, int n) {
        System.out.println("X#myst: int");
        return m;
    }
}


public class GenericMethods {
    public static void main(String[] args) {
        Mensch m = new Mensch();

        m.myst("Hello World", "m");
        m.myst("Hello World", 1);
        m.myst(3, 4);
        m.myst(m, m);
        m.<Mensch>myst(m, m);
        m.myst(m, 1);
        m.myst(3.0, 4);
```

```
38            m.<Double>myst(3, 4);
39        }
40    }
```

10 Wrap-Up

- Begriffe:
  - Generischer Typ: `Stack<T>`
  - Formaler Typ-Parameter: `T`
  - Parametrisierter Typ:`Stack<Long>`
  - Typ-Parameter: `Long`
  - Raw Type: `Stack`
- Generische Klassen: `public class Stack<E> { }`
  - "`<Typ>`" hinter Klassennamen
- Generische Methoden: `public <T> T foo(T m){ }`
  - "`<Typ>`" vor Rückgabewert

11 Zum Nachlesen

- Ullenboom (2021, Kap. 11.1)
- Oracle Corporation (2025)
- Oracle Corporation (2024)
- Bloch (2018)

Lernziele

- k1: Begriffe generischer Typ, parametrisierter Typ, formaler Typ-Parameter, Typ-Parameter
- k3: Erstellen und Nutzen von generischen Klassen und Interfaces
- k3: Erstellen und Nutzen von generischen Methoden

Quellen

Bloch, J. 2018. *Effective Java*. 3. Aufl. Addison-Wesley.
Oracle Corporation. 2024. „The Java Tutorials". 2024. https://docs.oracle.com/javase/tutorial/.
———. 2025. „Learn Java". 2025. https://dev.java/learn/.
Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 09509bb (tooling: shift headings (use h1 as top-level headings) (#993), 2025-04-23)